

Stacks Study Guide

Stacks Study Guide

1. Overview

A **stack** is a Last-In-First-Out (LIFO) data structure used extensively in algorithms involving parsing, backtracking, expression evaluation, and monotonic structures. Stacks appear in many LeetCode problems because they model real-world processes like undo operations, call stacks, and nested structures.

Stacks are ideal when you need:

- Reversals
 - Matching pairs (parentheses, tags)
 - Tracking previous states
 - Monotonic behavior (increasing/decreasing sequences)
-

2. Core Stack Patterns

2.1 Parentheses Matching

Used when validating or parsing nested structures.

Examples:

- Valid Parentheses
- Minimum Add to Make Parentheses Valid

Key idea:

```
for char in s:  
    if char is opening: push  
    else: check top of stack
```

2.2 Monotonic Stack

Used to maintain a stack that is strictly increasing or decreasing.

Examples:

- Daily Temperatures
- Next Greater Element
- Largest Rectangle in Histogram

Key idea:

```
while stack not empty and current breaks monotonic rule:  
    pop and process  
    push current
```

2.3 Expression Evaluation

Used for evaluating arithmetic expressions.

Examples:

- Basic Calculator I/II
- Evaluate Reverse Polish Notation

Key idea:

```
for token in expression:  
    if number: push  
    if operator: pop operands, compute, push result
```

2.4 Backtracking / Undo

Used when you need to revert to a previous state.

Examples:

- Browser history
 - DFS recursion stack
-

3. Essential Techniques

3.1 Using a List as a Stack

Python example:

```
stack = []  
stack.append(x) # push  
stack.pop() # pop  
stack[-1] # peek
```

3.2 Monotonic Increasing Stack Example

```
def nextGreaterElements(nums):
    res = [-1] * len(nums)
    stack = []

    for i, n in enumerate(nums):
        while stack and nums[stack[-1]] < n:
            res[stack.pop()] = n
        stack.append(i)

    return res
```

3.3 Largest Rectangle in Histogram (Classic Hard)

```
heights.append(0)
stack = [-1]
max_area = 0

for i, h in enumerate(heights):
    while heights[stack[-1]] > h:
        height = heights[stack.pop()]
        width = i - stack[-1] - 1
        max_area = max(max_area, height * width)
    stack.append(i)
```

4. Common Pitfalls

- Forgetting to check for empty stack before popping
- Misunderstanding monotonic direction
- Not handling edge cases (e.g., sentinel values)
- Incorrect width calculation in histogram problems
- Using stack when queue or deque is more appropriate

5. Practice Roadmap

Beginner

- Valid Parentheses
- Implement Stack Using Queues

- Min Stack

Intermediate

- Next Greater Element I/II
- Daily Temperatures
- Remove All Adjacent Duplicates

Advanced

- Largest Rectangle in Histogram
- Maximal Rectangle
- Basic Calculator II

6. Key Takeaways

- Stacks model LIFO behavior and are essential for parsing and monotonic problems.
- Monotonic stacks are powerful for next-greater/smaller problems.
- Many hard problems reduce to stack-based interval processing.

End of document.